

MODULE 22 · PART 7 - PROOF AND PERFORMANCE

Cost and Incrementality

The third question: not is it right, not is it fast, but is it worth what it costs.

PART 7 of 8 MODULE 22 of 26 RUNS about 30 min

Why cost is a separate question

- ◆ **Module twenty-one's method finds slow queries.** Nothing in this module is slow, and that is the point.
- ◆ **The question here is frequency,** not duration. How often is this computed against how often its answer changes.
- ◆ **Which no plan can tell you,** because a plan describes one execution and the problem is the number of them.
- ◆ **So the tool is arithmetic rather than EXPLAIN.** Count the consumers, count the changes, divide.

THE TAKEAWAY

A slow query log is a list of durations. The cost question needs a list of frequencies, and almost nobody has one.

The expensive query is rarely the problem.
The cheap query run four hundred times
usually is.

Where the money actually goes

- ◆ **The same aggregate computed by eleven dashboards**, each one cheap, none of them aware of the other ten.
- ◆ **A nightly rebuild of a table where 0.2% of the rows changed**, which is most nightly rebuilds.
- ◆ **A job that runs hourly because somebody once needed it hourly**, feeding a report read once a week.
- ◆ **None of those shows up in a slow query log**, because not one of them is slow.

THE TAKEAWAY

Module twenty-one's method finds slow queries. This module's question is different: what is being computed more often than the answer changes?

The multiplier, worked

```
1 a 30-second aggregate
2   x 14 dashboards that each compute it
3   x 200 opens per dashboard per day
4   = 2,800 computations per day
5   = 23 hours of compute, daily
6
7 the data it reads changes
8   x 1 time per day
9
10 -- 2,800 computations of a value that changes once.
```

5 Twenty-three hours of compute a day, from a query nobody would ever describe as slow.

THE TAKEAWAY

Nothing here appears in a slow query log. The largest single cost in many warehouses is a fast query with a large multiplier and no owner.

Finding the multiplier

- ◆ **Group your query history by a normalised query text**, and sort by total time rather than by longest.
- ◆ **Sorting by longest finds the nightly job**, which everybody already knows about and which runs once.
- ◆ **Sorting by count times duration finds the multiplier**, and it is usually something nobody has looked at.
- ◆ **Every warehouse has this history**, in a system view, and almost nobody aggregates it this way.

THE TAKEAWAY

One query against the query log, sorted by total rather than maximum. It reliably surfaces something surprising, and it takes about five minutes.

Process **what changed**, not everything.
Simple to say, and it has one hard
problem.

Reading a cost question out loud

WHAT YOU HAVE	WHAT TO ASK
a nightly rebuild	how many rows actually changed?
a 3-day lookback	what is the 99th percentile of the arrival lag?
an aggregate in 14 dashboards	how often does the answer change?
an hourly job	when was this last read?
a materialised table	how many things read it, and is it still one definition?

THE TAKEAWAY

Five questions, and each one is answerable with a single query. The reason they go unasked is that nothing prompts them, because nothing is failing.

The naive incremental

```
1 -- rebuild yesterday only
2 DELETE FROM agg_ledger
3   WHERE effective_date = CURRENT_DATE - 1;
4
5 INSERT INTO agg_ledger
6   SELECT effective_date, store_key, SUM(amount)
7     FROM fact_ledger
8    WHERE effective_date = CURRENT_DATE - 1
9    GROUP BY effective_date, store_key;
10
11 -- fast, idempotent, and wrong.
```

11 It assumes a posting for yesterday arrived yesterday. On Kestrel's ledger, 14% of them did not.

THE TAKEAWAY

The structure is right: delete the window, then insert it. The window is wrong, and the window is the entire difficulty.

Compute a thing **as often as its answer changes**, and no more often than that.

How much it misses, measured

```
1 fact_ledger: 400,000 postings
2 back-dated:      55,873 (14.0%)
3 median lag:      22 days
4 maximum lag:     44 days
5
6 lookback  captured  missed
7 1 day      86.03%    55,873
8 7 days     87.94%    48,254
9 14 days    90.16%    39,365
10 30 days   95.24%    19,047
11 45 days   100.00%     0
```

8 A week's lookback sounds generous and captures 2% more than one day, because the median lag is 22 days.

THE TAKEAWAY

Measured on the capstone database. The curve is almost flat until day 14 and then climbs steeply, which is the opposite of what a lookback window is usually sized for.

Why the intuition fails

- ◆ **A week feels generous**, and it is sized against how late you imagine data being, not against how late it is.
- ◆ **The lag distribution is not concentrated near zero here**. The median is 22 days, so half the late data is beyond three weeks.
- ◆ **So size the window from the distribution**, which is one query, rather than from a number that sounds safe.
- ◆ **And the distribution is a business fact**, not a technical one. It is how long the accounting process takes.

THE TAKEAWAY

One query gives you the lag distribution. Almost nobody runs it, and almost every lookback window in production was chosen by feel.

The query nobody runs

```
1 SELECT posted_ts::DATE - effective_date AS lag_days,  
2        COUNT(*) AS postings,  
3        SUM(COUNT(*)) OVER (ORDER BY posted_ts::DATE - effective_date  
4                               ROWS UNBOUNDED PRECEDING) * 1.0  
5        / SUM(COUNT(*)) OVER () AS cumulative_share  
6 FROM fact_ledger  
7 GROUP BY lag_days  
8 ORDER BY lag_days;
```

- 5 A running share, which is module seven's doubled aggregate. Read down it to the share you need and that row is your window.

THE TAKEAWAY

Eight lines, run once. It converts a lookback window from a guess into a decision with a stated confidence, and it is the same technique as module eight's Pareto.

A ledger aggregate is rebuilt incrementally with a three-day lookback, chosen when the pipeline was written. Month-end figures are restated for weeks after close, and finance have stopped trusting the mid-month numbers.

The ledger back-dates postings. Nobody has measured by how much. The pipeline produces no errors and the three-day window has never been revisited.

What is the first thing to do?

- A Measure the lag distribution, then size the window from it. Three days was chosen by feel and may be wildly short.
- B Switch to a full rebuild every night, which removes the window and therefore removes the problem entirely.
- C Extend the lookback to seven days, which is the conventional choice and roughly doubles the coverage.
- D Ask the source system to stop back-dating, since a posting dated in the past is a data quality problem.

◆ PAUSE AND DECIDE

What is the first thing to do?

- A

 Measure the lag
One query. On this data it shows a median lag of 22 days, which makes three days and seven days equally inadequate.
- B

 Full rebuild
Correct and it treats the window as unknowable. Measured here, a correctly sized window was 17 times cheaper and equally correct.
- C

 Seven days
Another number chosen by feel. Measured on this data it captures 2% more than one day, because the lag is not concentrated near zero.
- D

 Stop back-dating
Back-dating is the accounting process, not a defect. A posting effective in March that was approved in April is correctly dated to March.

THE SIGNAL

"Nobody has measured by how much." That sentence is the finding. Every option except A is another guess, and one of them is a guess that happens to be safe.

Cheap and wrong, or correct and expensive. **A correctly sized window is both correct and cheap.**

Both correct, and one is 17x cheaper

```
1 full rebuild          400,000 rows read    0.121s
2 45-day window         31,262 rows read    0.007s
3
4 -- both capture 100% of the postings
5 -- and the window costs 7.7% of the table per night
6
7 -- the 1-day window reads 684 rows
8 -- and misses 55,873 postings
```

- 2 Measured. The window is correct because it was sized from the data, and cheap because it is still only 7.7% of the table.

THE TAKEAWAY

The correctly sized window is not a compromise between the two options. It is better than both, and finding it took one query about the lag distribution.

The window, as a parameter

```
1 -- not this
2 WHERE effective_date >= CURRENT_DATE - 3
3
4 -- this
5 WHERE effective_date >= CURRENT_DATE - :lookback_days
6
7 -- with, somewhere a person will find it:
8 --   lookback_days = 45
9 --   measured 2026-06-01: 45 days captures 100% of postings,
10 --   median arrival lag 22 days, max 44. Re-measure annually.
```

- 9 The number, the date it was measured, and what it was measured against. Without those it becomes folklore within a year.

THE TAKEAWAY

A hard-coded 3 tells the next person nothing. A parameter with a measurement date and a stated coverage tells them when to check it and how.

Where the window should live

- ◆ **In one place, named**, so that changing it is one edit and every job using it moves together.
- ◆ **Not repeated across seven jobs**, which is module fourteen's session threshold problem in a different clause.
- ◆ **And it should be asserted**, by checking that nothing arrives with a lag beyond it, which is one query per night.
- ◆ **That assertion is the one that tells you the business changed**, before finance does.

THE TAKEAWAY

A nightly check that no posting arrived with a lag beyond the window converts a silent correctness failure into an alert, and it costs one aggregate.

The window assertion

```
1 SELECT
2     MAX(posted_ts::DATE - effective_date)      AS max_lag_seen,
3     :lookback_days                            AS window_days,
4     MAX(posted_ts::DATE - effective_date)
5     < :lookback_days                          AS window_still_adequate
6 FROM fact_ledger
7 WHERE posted_ts >= CURRENT_DATE - 7;
```

7 Only the last week's arrivals, so it reflects the process as it is now rather than as it was three years ago.

THE TAKEAWAY

Module twenty's termination assertion, in a pipeline. It checks that a guard you set is still bigger than the thing it guards against.

An incremental load must be **rerunnable**.
Pipelines fail, and somebody will run it
twice.

Three ways to be idempotent

```
1 -- 1. delete then insert the window
2 DELETE FROM agg WHERE effective_date >= :from;
3 INSERT INTO agg SELECT ... WHERE effective_date >= :from;
4
5 -- 2. replace whole partitions
6 INSERT OVERWRITE TABLE agg PARTITION (dt) -- Spark, Hive
7
8 -- 3. MERGE on the key
9 MERGE INTO agg t USING new s ON t.k = s.k
10   WHEN MATCHED THEN UPDATE SET ...
11   WHEN NOT MATCHED THEN INSERT ...;
12
13 -- and INSERT alone, which is not idempotent at all.
```

13 The default, and the one most pipelines start with. It works until the first rerun.

THE TAKEAWAY

The first is the simplest and it is what most warehouses should use. MERGE is more precise, more expensive, and needs a key that genuinely identifies a row.

What makes delete-then-insert safe

- ◆ **The delete and the insert use the same window**, so whatever is removed is exactly what is put back.
- ◆ **And they must be in one transaction**, or a failure between them leaves a hole that looks like missing data.
- ◆ **Which some warehouses do not offer**, so the window is briefly empty and a reader during it sees a gap.
- ◆ **Where that matters, write to a new table and swap**, which most engines can do atomically.

THE TAKEAWAY

The failure between the delete and the insert is the one people meet in production. It leaves a correctly shaped table missing three days, and no error anywhere.

A nightly incremental load appends the last day's aggregates to a summary table. It failed at 03:00 on Tuesday after the insert had partially completed, and was rerun manually at 08:00.

Tuesday's figures are now roughly 1.6 times the correct value. Monday and Wednesday are correct. The load produced no errors on either run.

What is the fix?

- A Add a check that the load has not already run today, so a rerun is refused rather than duplicating.
- B Deduplicate the summary table after every load, which removes whatever the rerun added.
- C Delete the window before inserting it, in one transaction, so a rerun replaces rather than appends.
- D Make the pipeline resume from where it failed, so the partially completed insert is not repeated.

◆ PAUSE AND DECIDE

What is the fix?

A Refuse a rerun

Blocks the recovery from a genuine failure. The Tuesday run needed to happen; it needed to not duplicate.

C Delete then insert

A rerun replaces the window rather than adding to it, so running it once, twice or ten times gives the same table.

B Deduplicate after

Module four's deduplication with no stated key and no ordering. It also cannot tell a duplicate from two legitimately equal rows.

D Resume from failure

Requires knowing precisely how far the insert got, which is the hard problem. Idempotence makes that question unnecessary.

THE SIGNAL

"Roughly 1.6 times." Not exactly double, because the first run got partway through. That ratio is the signature of an append that was interrupted and repeated.

Incrementality needs **a way to know what changed**. Many tables do not have one.

What counts as a watermark

- ◆ **A load timestamp set by the warehouse**, which is the most reliable, because you control it.
- ◆ **A modified timestamp set by the source**, which is only as trustworthy as the source's discipline about setting it.
- ◆ **A change data capture stream**, which tells you exactly what changed, including deletes, which nothing else does.
- ◆ **Not the business date**. That is what Kestrel's ledger proves: the effective date says nothing about when a row arrived.

THE TAKEAWAY

The last point is the whole of the back-dating problem. A business date and an arrival time are different columns, and incrementality depends on the second.

The deletes problem

- ◆ **A watermark finds inserts and updates.** It cannot find a row that is no longer there.
- ◆ **So an incremental load silently keeps deleted rows forever,** and the table drifts upward from the source.
- ◆ **Change data capture solves it,** and a periodic full reconciliation catches it, and nothing else does.
- ◆ **Which is one good reason to rebuild fully sometimes,** on a cadence, as a correction rather than as the normal path.

THE TAKEAWAY

An incremental table that has never been fully rebuilt has been accumulating deleted rows for as long as it has existed, and no assertion inside it can notice.

A team is asked to make a 90-minute nightly dimension rebuild incremental, to cut cost. The dimension is updated in place by the source system, which overwrites rows and has no modified timestamp.

Rows are also occasionally deleted when a product is withdrawn. The source team cannot change the schema this year.

What should the review conclude?

- A Compare a hash of each row against the previous load to detect changes, which works without a source timestamp.
- B Add a warehouse-side load timestamp on ingest, which gives the watermark the source does not provide.
- C Process incrementally on the product key range, since new products get higher keys than existing ones.
- D Incrementality is not possible here. There is no way to know what changed, and deletes are invisible to any watermark.

◆ PAUSE AND DECIDE

What should the review conclude?

A Hash comparison

It detects changes and deletes, and it requires reading every row of the source to compute the hashes. That is the full rebuild it was meant to avoid.

C Key range

Finds new products and misses every update to an existing one, which is the majority of what changes in a dimension.

B A load timestamp

Records when the warehouse saw a row, and the warehouse sees every row every night because the source overwrites. Every row looks changed.

D Not possible here

No watermark exists, deletes are invisible, and hashing needs a full read. The honest answer, and it redirects the effort.

THE SIGNAL

"Updated in place, no modified timestamp, and rows are deleted." Three separate reasons, any one of which is sufficient. The conclusion is not a failure of effort.

Incremental pipelines are cheap to run and **expensive to change**. The bill arrives as backfills.

Backfills, and why they hurt

- ◆ **A backfill is a full rebuild you do under time pressure**, with a definition that has just changed and is not yet trusted.
- ◆ **It has to coexist with the nightly run**, which is where the idempotence matters most and is usually tested least.
- ◆ **And it is unbudgeted**, because the pipeline's cost was justified by the incremental nightly figure.
- ◆ **So count backfills per year when deciding**. Four a year on a table that rebuilds in an hour is four hours, against 365 nightly savings.

THE TAKEAWAY

The arithmetic usually still favours incremental. It is worth doing rather than assuming, because a table whose logic changes monthly is a different calculation.

Assertions for an incremental load

```
1 -- 1. the window was fully replaced, not appended to
2 COUNT(*) = COUNT(DISTINCT (effective_date, store_key))
3
4 -- 2. nothing arrived beyond the window
5 MAX(posted_ts::DATE - effective_date) < :lookback_days
6
7 -- 3. the incremental table still agrees with the source
8 SUM(amount) = (SELECT SUM(amount) FROM fact_ledger)
9
10 -- 4. and run 3 against ALL of history, weekly, not nightly
```

10 The nightly version only checks the window. The weekly one checks the history the window never revisits.

THE TAKEAWAY

Assertion three run only over the window cannot see drift in the history behind it, which is exactly where deleted rows and missed back-dates accumulate.

The weekly full reconciliation

- ◆ It is the only thing that finds a **deleted row**, because no watermark will ever surface one.
- ◆ And the only thing that finds a **missed back-date**, if the window was ever too short.
- ◆ It costs **one full read, weekly**, which is 1/7th of what a nightly rebuild would have cost anyway.
- ◆ So you get most of the **safety of a full rebuild for a seventh of the price**, which is the trade worth taking.

THE TAKEAWAY

Incremental nightly, reconcile fully weekly. It is the arrangement that captures both the cost saving and the correctness, and it is not the default anywhere.

Module ten asked whether to materialise.
The answer is a number: how many things read it.

The materialisation arithmetic

SITUATION	WHAT TO DO
One consumer, read once per build	Do not materialise. It is a CTE.
Eleven consumers, each recomputing	Materialise. Eleven computations become one.
Read constantly, changes hourly	Materialise, and refresh hourly. Staleness is the cost.
Read rarely, changes constantly	Do not. You would refresh far more often than anybody reads.
Expensive to compute, cheap to store	Materialise, almost regardless of the read count.

THE TAKEAWAY

The ratio that decides it is reads to changes. Materialising is right when a result is read more often than it changes, and wrong when it is the other way round.

Does the full rebuild still fit?

```
1 -- the check to run before engineering anything
2
3 full rebuild duration      0.121s
4 available window          the load slot, minus what else runs in it
5 projected growth          rows/day x days until you would revisit
6
7 -- if the rebuild finishes in the slot, with headroom for a year,
8 -- incremental is optimising a thing that is not a problem.
```

7 With headroom for a year, because a rebuild that fits today and not in March is a worse position than either.

THE TAKEAWAY

A great many pipelines were made incremental while the full rebuild comfortably fitted, buying nightly minutes with backfills, deletes and rerun risk.

Per-query billing changes the arithmetic

- ◆ **On a bytes-scanned engine, every read has a price tag**, so the multiplier is literally a bill rather than a metaphor.
- ◆ **Which makes partition pruning a cost control**, not a performance tweak. Module twenty-one's lever, with an invoice attached.
- ◆ **And materialising a small aggregate is unambiguously right**, because 2,800 scans of a fact table become 2,800 scans of a tiny one.
- ◆ **On a provisioned cluster the signal is weaker**, because the cost is fixed and the waste shows up as queueing instead.

THE TAKEAWAY

The two pricing models make the same waste visible in different ways. One bills you for it and the other makes your colleagues wait for it.

The cost of a dashboard, written down

```
1 measure:      net sales by region, month to date
2 computed in:  14 dashboards, independently
3 opens/day:    ~200 each
4 duration:     30s
5 source changes: once nightly
6
7 -> 2,800 computations/day of a value that changes once
8 -> 23 hours of compute daily
9 -> 14 definitions, which have never been compared
```

- 9 The line that wins the argument. The cost is large and the inconsistency is what somebody will eventually be embarrassed by.

THE TAKEAWAY

Nine lines, and anybody can read them. A cost case made in compute-hours is arguable; one that also says 'fourteen definitions' usually is not.

What materialising actually costs

- ◆ **Storage**, which is usually the cheapest part and the one people think of first.
- ◆ **Staleness**, because the stored answer is now as old as its last refresh, and somebody will not know that.
- ◆ **A thing to maintain**, which fails, needs monitoring, and has to be rebuilt when its definition changes.
- ◆ **And a dependency**, because eleven consumers now break together when it breaks.

THE TAKEAWAY

The fourth is the one that is underestimated. Materialising converts eleven independent slow queries into one shared point of failure, which is usually worth it and is a real change.

The things worth materialising in this course

- ◆ **Trading spells.** 237,000 store-days become 475 rows, read by half a dozen reports. Module thirteen.
- ◆ **Sessions.** 2.6 million events become 399,822 rows, and the threshold is then applied once. Module fourteen.
- ◆ **Price validity intervals.** 259,246 periods, built by one LEAD, read by every as-at join. Module fifteen.
- ◆ **All three have the same shape:** a large source, a small result, and several consumers who would otherwise disagree.

THE TAKEAWAY

In every case the correctness argument and the cost argument point the same way, which is rare and is how you know it is the right call.

A 30-second aggregate is computed independently inside fourteen dashboards. Each dashboard is opened around 200 times a day. The underlying data is loaded once a night.

A proposal is on the table to materialise it as a nightly table. An objection is raised that the aggregate is fast and the table would be another thing to maintain.

What should the decision be?

- A Leave it. A 30-second query is not a performance problem, and fourteen copies of it is a documentation problem instead.
- B Materialise it. It is recomputed roughly 2,800 times a day for data that changes once, and fourteen definitions become one.
- C Materialise it only for the three highest-traffic dashboards, leaving the others to compute it as they do now.
- D Cache the result at the dashboard layer instead, which avoids adding a table to the warehouse.

◆ PAUSE AND DECIDE

What should the decision be?

A Leave it

The objection is right that it is not slow, and wrong about what the problem is. 2,800 computations of a value that changes once is the cost.

C Three dashboards only

Now there are two definitions of the measure, and the eleven that were left behind will drift from the three that were not.

B Materialise

Reads vastly exceed changes, which is the ratio that decides it. And fourteen definitions collapsing to one is the larger win.

D Cache in the dashboard layer

Fixes the recomputation and leaves fourteen definitions in place. The consistency problem survives entirely.

THE SIGNAL

"Computed independently inside fourteen dashboards." The cost is the multiplier and the real prize is that fourteen slightly different definitions become one.

Two columns, two purposes

```
1 effective_date    when it happened    -> what the report groups by
2 posted_ts        when we learned it   -> what the pipeline filters on
3
4 -- the incremental window uses posted_ts
5 WHERE posted_ts >= :last_watermark
6
7 -- the report uses effective_date
8 GROUP BY effective_date
9
10 -- using one column for both is the whole back-dating bug.
```

10 Module fifteen's two timelines, arriving as a pipeline design decision rather than a query one.

THE TAKEAWAY

If the pipeline filtered on posted_ts rather than effective_date, no lookback window would be needed at all: every newly arrived row is selected regardless of its date.

So why is a lookback ever needed

- ◆ **Because the aggregate is grouped by the business date**, so a late row changes a day you already summarised.
- ◆ **Select by arrival, but rebuild by business date**. Find the affected dates, then rebuild exactly those.
- ◆ **Which is better than a fixed window**, because it rebuilds only the days that actually changed, however old.
- ◆ **And it needs the arrival column**. Without one you are back to guessing a window.

THE TAKEAWAY

Select by arrival, group by business date, rebuild the affected dates. It handles a 400-day-old correction correctly and touches nothing it does not have to.

Select by arrival, rebuild by date

```
1 -- 1. which business dates did tonight's arrivals touch?
2 CREATE TEMP TABLE affected AS
3 SELECT DISTINCT effective_date FROM fact_ledger
4 WHERE posted_ts >= :last_watermark;
5
6 -- 2. rebuild exactly those, however old they are
7 DELETE FROM agg WHERE effective_date IN (SELECT * FROM affected);
8 INSERT INTO agg
9 SELECT effective_date, store_key, SUM(amount) FROM fact_ledger
10 WHERE effective_date IN (SELECT * FROM affected)
11 GROUP BY effective_date, store_key;
```

7 No window, no guess, and no maximum age. A correction to a date two years ago rebuilds that date and nothing else.

THE TAKEAWAY

This is the correct pattern and it needs an arrival column. Where one exists, the lookback window and the whole lag-distribution question simply go away.

Which pattern to use

- ◆ **Arrival-driven, if you have an arrival column.** Exact, no window to size, and it handles any age of correction.
- ◆ **A sized lookback, if you do not,** measured from the lag distribution rather than chosen by feel.
- ◆ **A full rebuild, if it fits,** which is simpler than both and free to change.
- ◆ **And never an unmeasured window,** which is the option most pipelines are running today.

THE TAKEAWAY

Three good options and one bad one. The bad one is the default, because it is what you get by writing the obvious thing and never revisiting it.

Cheap to run, or cheap to change. **They pull in opposite directions**, and the choice is rarely made consciously.

The trade, stated

	FULL REBUILD INCREMENTAL
Nightly cost	high low
Changing the logic	free: tomorrow uses it a backfill
Fixing a bug in history	free a backfill
Late-arriving data	handled needs a sized window
Deletes	handled invisible
Rerunning after a failure	safe only if idempotent

THE TAKEAWAY

Incremental wins one row of this table. It is usually the right call anyway, and it is worth knowing that you are buying nightly cost with five other things.

When a full rebuild is the right answer

- ◆ **When it fits in the window.** If a full rebuild finishes in time, it is simpler, safer and free to change.
- ◆ **When the logic is still moving,** because a table whose definition changes monthly will be backfilled monthly.
- ◆ **When deletes matter,** because no watermark will find them.
- ◆ **And not when it merely feels safer,** because rebuilding 400,000 rows to change 684 is a habit rather than a decision.

THE TAKEAWAY

The first point is the one to check before any of the engineering. A great many pipelines were made incremental while the full rebuild still comfortably fitted.

Deciding, in four questions

01

How often does it change?

Against how often it is read. That ratio decides materialisation on its own.

02

Can you know what changed?

A watermark, or CDC. Without one, incremental is not available at any price.

03

How late does data arrive?

Measure the lag distribution. It sizes the window, and it is one query.

04

Is it rerunnable?

Delete the window, then insert it, in one transaction. Assume it will be run twice.

THE TAKEAWAY

Question three is the one this module exists for. Every lookback window in production was either measured or guessed, and almost all of them were guessed.

Staleness, stated

- ◆ **A materialised table has an age**, and every consumer is entitled to know what it is.
- ◆ **Carry the refresh timestamp as a column**, so a report can display it and a check can alert on it.
- ◆ **Without it, staleness is invisible**, and a pipeline that silently stopped three days ago looks exactly like one that ran.
- ◆ **Which is the commonest data incident there is**: not wrong data, but yesterday's data presented as today's.

THE TAKEAWAY

A stopped pipeline produces a table that is entirely correct and three days old. Every assertion inside it passes, because nothing in it is wrong.

Every pipeline is **a cache**. The only questions are what it caches and how stale you will let it get.

Who owns the multiplier

- ◆ **Nobody, usually.** Each dashboard author owns a query that takes thirty seconds, which is entirely reasonable.
- ◆ **And the platform team owns a bill**, with no visibility of which query inside it is the multiplied one.
- ◆ **The query history joins the two**, which is why aggregating it by normalised text is worth doing quarterly.
- ◆ **This is an organisational problem wearing a technical costume**, and the technical part is one query.

THE TAKEAWAY

Nobody is behaving badly here. Fourteen people each wrote a reasonable query, and the cost is a property of the set rather than of any member of it.

What to do on Monday

- ◆ **Aggregate the query history by normalised text, sorted by total time.** Five minutes, and it usually surprises.
- ◆ **Measure the arrival lag on your largest incremental table,** and compare it with the window that is configured.
- ◆ **Check whether the biggest nightly rebuild still fits,** and whether it was ever incremental for a reason that still holds.
- ◆ **Three queries, one afternoon.** Between them they answer the three numbers from earlier in this module.

THE TAKEAWAY

None of these requires a project, a meeting or a budget. All three produce a number that either settles a question or starts a much better conversation.

Most pipelines are **correct, affordable and slightly wrong about one window**. The window is the thing to check.

Three numbers to know about any pipeline

- ◆ **How many rows changed against how many it processes.** If that ratio is tiny, it is doing avoidable work.
- ◆ **How often it runs against how often it is read.** If it runs more often, that ratio is pure waste.
- ◆ **And the arrival lag distribution,** because that decides whether the answers are correct at all.
- ◆ **Three queries.** Most teams can answer none of them about their largest pipeline.

THE TAKEAWAY

The first two are about cost and the third is about correctness, and all three are unanswered in most warehouses because nothing prompts anybody to ask.

What Part 7 gave you

MODULE	THE QUESTION IT ANSWERS
20 Proving a query right	How would I know if this were wrong?
21 Reading a plan	Where is the time, and what fed that operator?
22 Cost and incrementality	How often is this computed, against how often it changes?
all three	And they must be asked in that order.

THE TAKEAWAY

Right, then fast, then affordable. Optimising an unverified query is guessing, and reducing the cost of a wrong answer is worse than leaving it alone.

The confusions, settled

BELIEF	THE TRUTH
Incremental is cheaper	Usually, and it costs you backfills, deletes and rerun safety.
A week's lookback is generous	Measured here: 2% more than one day, with a 22-day median lag.
A full rebuild is the safe option	It is, and a correctly sized window was 17x cheaper and equally correct.
The expensive query is the cost	The cheap query run 2,800 times usually is.
Materialising costs storage	Storage is the cheap part. Staleness and a shared dependency are not.
Back-dated data is a quality problem	It is the accounting process. Size the window for it.

THE TAKEAWAY

Rows two and three are measured on the capstone ledger. The rest follow from asking how often a thing is computed against how often its answer changes.

Five habits

- ◆ **Measure the lag distribution before sizing any window**, because it is one query and the alternative is a guess.
- ◆ **Delete the window, then insert it, in one transaction**, and assume the job will be run twice.
- ◆ **Rebuild fully on a cadence**, as a correction, because deletes are invisible to every watermark.
- ◆ **Count the consumers before materialising**, and count the changes, because the ratio is the decision.
- ◆ **And check whether a full rebuild still fits**, before engineering an incremental one that does not need to exist.

THE TAKEAWAY

The last one saves the most effort and is checked the least. A pipeline made incremental while the rebuild still fitted has bought nightly minutes with years of backfills.

Right, fast, affordable. **Part 8** is where you find out whether any of it stuck.